



Web Development Lab

Responsive Sidebar Navigation System — Complete Practical Session Guide

18 Guided Steps • Beginner–Intermediate • Estimated Time: 50 mins

Lab Overview

In this lab session, students will build a **fully responsive Sidebar Navigation System** using:

- HTML5 Semantic Layout
- CSS Flexbox
- CSS Positioning
- Media Queries
- JavaScript DOM Manipulation
- Event Bubbling and Propagation

Students will learn how modern dashboards are built in:

- SaaS Platforms
- Learning Management Systems
- Admin Dashboards
- Project Management Applications

This practical introduces several important frontend engineering concepts:

- Responsive Layout Design
- Sidebar Toggle Systems
- Mobile Navigation Patterns
- Dynamic Class Manipulation

- Event Listeners
- Component-Based UI Structuring

The complete sidebar HTML structure used in this lab is available on the google classroom

Prerequisites

Before beginning this lab, students should already understand:

- Basic HTML Structure
- CSS Selectors
- Flexbox Basics
- Basic JavaScript Syntax
- CSS Positioning

1. Create the Project Structure

Objective: Set up the project files and structure.

🕒 5 min

Background

A modern frontend project is usually divided into:

- HTML → Structure
- CSS → Styling
- JavaScript → Interactivity

This separation improves:

- Maintainability
- Scalability
- Readability

Instructions

Continue working on your default folder structure

2. Inside the folder create:

- `sidebar.html`
- `sidebar.css`
- `triggersidebar.js`

2. Build the HTML Boilerplate

Objective: Create the foundational HTML structure.

🕒 5 min

Background

- Every HTML document starts with a boilerplate structure.

- `<!DOCTYPE html>`

- Declares HTML5 document type

- `<meta charset="UTF-8">`

- Supports modern text encoding

- `<meta name="viewport">`

- Makes the website responsive on mobile devices

- `<link rel="stylesheet">`

- Connects the external CSS file

Instructions

1. Add the following HTML structure to `index.html`

Code Snippet

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sidebar</title>
  <link rel="stylesheet" href="sidebar.css">
</head>
<body>

</body>
</html>
```



Tip: The viewport meta tag is essential for responsive web design.

3. Create the Main Layout Container

Objective: Build the overall page structure.

🕒 6 min

Background

- The layout contains:
 - Sidebar Section
 - Main Content Section
- Acts as the parent wrapper for the entire page layout
- Semantic HTML element for secondary navigation
- Semantic HTML element for primary content

```
■ <div class="layout">
```

```
■ <aside>
```

```
■ <main>
```

Instructions

1. Add the layout container

Code Snippet

```
<div class="layout">

  <aside class="sidebarContainer">
  </aside>

  <main class="mainContent">
  </main>

</div>
```



Tip: Semantic HTML improves accessibility and SEO.

4. Reset Browser Default Styling

Objective: Remove default browser spacing.

🕒 5 min

Background

- Browsers automatically apply:
 - Margins
 - Padding
 - This causes inconsistent layouts across browsers.
 - Universal selector targets ALL elements

- `Margin:0;`

- Removes external spacing

- `Padding:0;`

- Removes internal spacing

Instructions

1. Open `sidebar.css`
2. Add the reset styling

Code Snippet

```
* {  
  margin:0;  
  padding:0;  
}
```

💡 **Tip:** CSS resets ensure consistency across browsers.

5. Convert the Layout into a Flex Container

Objective: Position sidebar beside the main content.

🕒 6 min

Background

Flexbox allows horizontal alignment.

- `Display: flex;`
 - Places children side-by-side horizontally
- `Min-height: 100vh;`
 - Makes layout occupy full screen height

Instructions

1. Style `.layout`

Code Snippet

```
.layout{  
  display: flex;  
  min-height: 100vh;  
}
```



Tip: Sidebar layouts are commonly built with Flexbox.

6. Build the Sidebar Container

Objective: Create a fixed-width sidebar.

🕒 7 min

Background

- `background-color:#1D293D;`
 - Applies dark dashboard styling
- `Flex-shrink:0;`
 - Prevents sidebar from shrinking
- `min-width:255px;`
- `Max-width:255px;`
 - Locks sidebar width

Instructions

1. Style `.sidebarContainer`

Code Snippet

```
.sidebarContainer{  
  background-color: #1D293D;  
  flex-shrink: 0;  
  min-width: 255px;  
  max-width: 255px;  
}
```



Tip: Fixed-width sidebars improve navigation consistency.

7. Create the Main Content Area

Objective: Allow content to fill remaining space.

🕒 5 min

Background

- `flex-grow:1;`
- Allows the content area to expand
- `background-color:#020618;`
- Creates dashboard-style dark background

Instructions

1. Style `.mainContent`

Code Snippet

```
.mainContent{
  flex-grow:1;
  background-color: #020618;
  color:white;
}
```

💡 **Tip:** `flex-grow` is critical for responsive layouts.

8. Build the Sidebar Header Section

Objective: Align logo and company information.

🕒 7 min

Background

- `Display: flex;`
 - Places items horizontally
- `Gap: 12px;`
 - Adds spacing between items
- `Padding: 24px;`
 - Adds internal spacing

Instructions

1. Add the sidebar header HTML
2. Style `.sideBarHeader`

Code Snippet

```
<section class="sideBarHeader">

  <div>
    
  </div>

  <div>
    <p>CSS Teaching</p>
    <small>Interactive Examples</small>
  </div>

</section>

.sideBarHeader{
  display: flex;
  padding: 24px;
  gap:12px;
}
```

 **Tip: Flexbox is ideal for logo + text alignment.**

9. Style Sidebar Header Typography

Objective: Improve visual hierarchy.

🕒 6 min

Background

- `Font-weight:bold;`
 - Makes text prominent
- `Font-size:18px;`
 - Controls text size
- `color:#FFFFFF;`
 - Applies white text color

Instructions

1. Style `.sideBarHeader p`
2. Style `.sideBarHeader small`

Code Snippet

```
.sideBarHeader p{
  font-weight: bold;
  font-size: 18px;
  color:#FFFFFF;
}

.sideBarHeader small{
  font-size: 12px;
  color:#99A1AF;
}
```



Tip: Typography establishes hierarchy and readability.

10. Create the Overview Section

Objective: Build the top navigation item.

🕒 7 min

Background

- `Display:flex;`
 - Aligns icon and text horizontally
- `Align-items:center;`
 - Centers items vertically
- `Gap:12px;`
 - Adds spacing between icon and label

Instructions

1. Add the Overview HTML
2. Style `.overviewSection`

Code Snippet

```
.overviewSection {  
    display: flex;  
    align-items: center;  
    gap:12px;  
}
```



Tip: Navigation items usually combine icons with labels.

11. Build the Sidebar Links Section

Objective: Create reusable navigation items.

🕒 8 min

Background

- `Display: flex;`
 - Aligns icons and text horizontally
- `Margin-bottom: 20px;`
 - Adds spacing between navigation items
- `transition: background-color 0.2s ease;`
 - Creates smooth hover animations

Instructions

1. Style `.SidebarLinks li`

Code Snippet

```
.SidebarLinks li{
  padding: 10px 5px;
  display: flex;
  gap: 5px;
  margin-bottom: 20px;
  transition: background-color 0.2s ease;
}
```



Tip: Reusable navigation systems improve scalability.

12. Add Hover Effects to Sidebar Links

Objective: Improve interactivity.

🕒 5 min

Background

- `:hover`
 - Applies styles during hover
- `background-color:#80808080;`
 - Adds translucent hover background
- `Border-radius:5px;`
 - Rounds hover edges

Instructions

1. Add hover styling

Code Snippet

```
.SidebarLinks li:hover{  
    background-color: #80808080;  
    border-radius: 5px;  
}
```



Tip: Hover states improve user experience.

13. Create the Sidebar Toggle Button

Objective: Build the mobile navigation trigger.

🕒 6 min

Background

- `Cursor:pointer;`
 - Indicates interactivity
- `background:none;`
- `Border:0;`
 - Removes default browser button styling

Instructions

1. Add the toggle button HTML
2. Style `.sidebar-toggle`

Code Snippet

```
.sidebar-toggle {  
  width: 25px;  
  height: 25px;  
  cursor: pointer;  
  background: none;  
  border: 0;  
}
```



Tip: Hamburger buttons are standard in mobile navigation.

14. Build Responsive Media Queries

Objective: Hide sidebar on mobile devices.

🕒 10 min

Background

- `@media (max-width:768px)`
 - Applies styles only on smaller screens
- `Position:fixed;`
 - Locks sidebar relative to viewport
- `transform:translateX(-100%);`
 - Moves sidebar completely off-screen
- `transition:transform 0.3s ease-in-out;`
 - Animates sidebar movement smoothly

Instructions

1. Add responsive media query

Code Snippet

```
@media (max-width: 768px) {  
  .sidebarContainer {  
    position: fixed;  
    top: 0;  
    left: 0;  
    height: 100vh;  
    transform: translateX(-100%);  
    transition: transform 0.3s ease-in-out;  
  }  
}
```



Tip: Mobile-first responsiveness is industry standard.

15. Create the Sidebar Open State

Objective: Slide sidebar into view dynamically.

🕒 6 min

Background

- `.sidebarContainer.open`
 - Targets sidebar ONLY when `.open` exists
- `transform:translateX(0);`
 - Moves sidebar back into view

How the `.open` class works

The Timeline of how it works:

1. **The Page Loads:** The browser reads the HTML. It sees `.sidebarContainer` *does not* have the `.open` class. It applies the default CSS and hides the sidebar off-screen.
2. **The User Clicks:** The user clicks our toggle button.
3. **JavaScript Injects the Class:** The script instantly updates the HTML behind the scenes while the page is running. It forces the HTML to change to this:

```
<aside class="sidebarContainer open">
```

Instructions

1. Add `.open` styling

Code Snippet

```
.sidebarContainer.open {  
  transform: translateX(0);  
}
```

💡 **Tip:** Class-based UI states are common in modern frontend engineering.

16. Select Sidebar Elements Using JavaScript

Objective: Access sidebar components dynamically.

🕒 7 min

Background

- `document.querySelector()`
 - Selects HTML elements
- `const sidebarButton`
 - Stores toggle button
- `const sidebar`
 - Stores sidebar container

Instructions

1. Open `triggersidebar.js`
2. Add selectors

Code Snippet

```
const sidebarButton = document.querySelector(".sidebar-toggle");  
const sidebar = document.querySelector(".sidebarContainer");
```



Tip: DOM selection is foundational in JavaScript.

17. Toggle Sidebar Visibility

Objective: Open and close the sidebar dynamically.

🕒 8 min

Background

- `addEventListener("click")`
 - Listens for button clicks
- `classList.toggle("open")`
 - Adds/removes `.open` class dynamically
- `event.stopPropagation()`
 - Stops event bubbling

Without this:

- Sidebar opens
- Then instantly closes

Instructions

1. Add sidebar toggle logic

Code Snippet

```
sidebarButton.addEventListener("click", function(event) {  
    sidebar.classList.toggle("open");  
    event.stopPropagation();  
});
```



Tip: Event bubbling is a critical JavaScript concept.

18. Close Sidebar When Clicking Outside

Objective: Improve mobile navigation UX.

🕒 8 min

Background

- `document.addEventListener()`
 - Listens globally across page
- `closest()`
 - Checks if clicked element exists inside sidebar
- `classList.remove("open")`
 - Closes sidebar

Instructions

1. Add outside-click detection

Code Snippet

```
document.addEventListener("click", function(event) {  
  
    if (sidebar.classList.contains("open")) {  
  
        if (!event.target.closest(".sidebarContainer")) {  
            sidebar.classList.remove("open");  
        }  
    }  
});
```



Tip: Outside-click handling is used in modals, dropdowns, and sidebars.

19. Final Review and Testing

Objective: Validate the complete sidebar system.

🕒 10 min

Instructions

1. Open browser
2. Resize screen to mobile width
3. Click sidebar toggle button
4. Verify sidebar slides into view
5. Click outside sidebar
6. Verify sidebar closes

Final Check

- Sidebar responsive behavior works
- Sidebar toggle works properly
- Outside click detection works
- Layout scales across devices
- No JavaScript console errors
- No CSS overflow issues